CS 171: Problem Set 4

Due Date: October 10th, 2026 at 8:59pm via Gradescope

1. The DES Complementation Property (10 points)

The DES block cipher is a 16-round Feistel network with a block size of 64 bits. Its key schedule derives a sequence of 48-bit round keys $k_1, \ldots, k_{16}$ from the master key. The details of the key schedule are not important for this exercise.

The DES round function $\widehat{f}$ takes a 48-bit round key $k_i \in \{0,1\}^{48}$ and a 32-bit input $R \in \{0,1\}^{32}$. It is evaluated as follows:

1. The expansion function E expands R to 48 bits by duplicating certain bits of R.
2. The value $E(R)$ is XORed with k_i.
3. The resulting 48-bit string is divided into eight 6-bit blocks. Each block is passed through an S-box that maps 6 bits to 4 bits.
4. A fixed permutation is applied to the resulting 32-bit string.

For a bit string y, let $\overline{y}$ denote its bitwise complement. Prove that, for every sequence of round keys $k_1, \ldots, k_{16}$ and every input $x \in \{0,1\}^{64}$,

$$\mathsf{DES}_{k_1,\ldots,k_{16}}(x) = \overline{\mathsf{DES}_{\overline{k_1},\ldots,\overline{k_{16}}}(\overline{x})}.$$

Solution We first prove that the DES round function satisfies

$$\widehat{f}(k, R) = \widehat{f}(\overline{k}, \overline{R})$$

for every $k \in \{0,1\}^{48}$ and $R \in \{0,1\}^{32}$. Because the expansion function only copies and reorders bits,

$$E(\overline{R}) = \overline{E(R)}.$$

Therefore,

$$E(\overline{R}) \oplus \overline{k} = \overline{E(R)} \oplus \overline{k} = E(R) \oplus k.$$

Thus the inputs to all eight S-boxes are identical in the two computations. Their outputs are therefore identical, and applying the same fixed permutation preserves equality. This proves the claimed identity for $\widehat{f}$.

Now write the state before round i as (L_{i-1}, R_{i-1}). A Feistel round computes

$$L_i = R_{i-1}, \qquad R_i = L_{i-1} \oplus \widehat{f}(k_i, R_{i-1}).$$

Let (L'_i, R'_i) denote the state obtained when the initial input and every round key are complemented. We prove by induction on i that

$$L'_i = \overline{L_i} \qquad \text{and} \qquad R'_i = \overline{R_i}.$$

The claim holds before the first round because the initial input is complemented. Suppose it holds before round i. Then

$$L'_i = R'_{i-1} = \overline{R_{i-1}} = \overline{L_i}.$$

Using the identity for the round function,

$$\begin{aligned} R'_i &= L'_{i-1} \oplus \widehat{f}(\overline{k_i}, R'_{i-1}) \\ &= \overline{L_{i-1}} \oplus \widehat{f}(\overline{k_i}, \overline{R_{i-1}}) \\ &= \overline{L_{i-1}} \oplus \widehat{f}(k_i, R_{i-1}) \\ &= \overline{L_{i-1} \oplus \widehat{f}(k_i, R_{i-1})} \\ &= \overline{R_i}. \end{aligned}$$

The induction therefore holds through all 16 rounds. Consequently, every bit of the final state in the complemented computation is the complement of the corresponding bit in the original computation, which gives

$$\mathsf{DES}_{k_1,\ldots,k_{16}}(x) = \overline{\mathsf{DES}_{\overline{k_1},\ldots,\overline{k_{16}}}(\overline{x})}.$$

■

2. Insecure Candidates for MACs (10 points)

Two candidate MAC constructions are given below. Both use a pseudorandom function $F : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$. Show that each construction is insecure.

1. Scheme 1:
   (a) $\mathsf{Gen}(1^n)$: Sample $k \xleftarrow{\$} \{0,1\}^n$ and output k.
   (b) $\mathsf{Mac}(k, m)$: Parse $m = m_0 \,\|\, m_1$, where $m_0, m_1 \in \{0,1\}^n$, and output
   $$t = F(k, m_0) \,\|\, F(k, m_0 \oplus m_1).$$
   (c) $\mathsf{Verify}(k, m, t)$: Output 1 if $t = \mathsf{Mac}(k, m)$, and output 0 otherwise.
2. Scheme 2:
   (a) $\mathsf{Gen}(1^n)$: Sample $k \xleftarrow{\$} \{0,1\}^n$ and output k.
   (b) $\mathsf{Mac}(k, m)$: Parse $m = m_0 \,\|\, m_1$, where $m_0, m_1 \in \{0,1\}^{n-1}$. Sample $r \xleftarrow{\$} \{0,1\}^n$ and output
   $$t = r \,\|\, \big(F(k, r) \oplus F(k, 0 \,\|\, m_0) \oplus F(k, 1 \,\|\, m_1)\big).$$
   (c) $\mathsf{Verify}(k, m, t)$: Parse $m = m_0 \,\|\, m_1$, where $m_0, m_1 \in \{0,1\}^{n-1}$, and parse $t = r \,\|\, t'$, where $r, t' \in \{0,1\}^n$. Output 1 if
   $$t' = F(k, r) \oplus F(k, 0 \,\|\, m_0) \oplus F(k, 1 \,\|\, m_1),$$
   and output 0 otherwise.

Solution Scheme 1. Choose $m_0, m_1 \in \{0,1\}^n$ such that $m_0 \neq 0^n$ and $m_0 \neq m_1$. The adversary queries the following two messages:

$$m = m_0 \,\|\, m_0 \qquad \text{and} \qquad m' = 0^n \,\|\, (m_0 \oplus m_1).$$

It receives

$$\mathsf{Mac}(k, m) = F(k, m_0) \,\|\, F(k, 0^n),$$
$$\mathsf{Mac}(k, m') = F(k, 0^n) \,\|\, F(k, m_0 \oplus m_1).$$

The adversary can therefore combine the first half of the first tag with the second half of the second tag and output

$$m^* = m_0 \,\|\, m_1, \qquad t^* = F(k, m_0) \,\|\, F(k, m_0 \oplus m_1).$$

The tag t^* is valid for m^*. Moreover, $m^* \neq m$ because $m_0 \neq m_1$, and $m^* \neq m'$ because $m_0 \neq 0^n$. Thus m^* was never queried, and the adversary wins the MAC security game with probability 1.

Scheme 2. The adversary chooses any $m = m_0 \,\|\, m_1$ and queries its MAC oracle on m. It receives $t = r \,\|\, t'$, where

$$t' = F(k, r) \oplus F(k, 0 \,\|\, m_0) \oplus F(k, 1 \,\|\, m_1).$$

Write $r = r_0 \,\|\, r'$, where $r_0 \in \{0,1\}$ and $r' \in \{0,1\}^{n-1}$.

If $r_0 = 0$, the adversary outputs

$$m^* = r' \,\|\, m_1, \qquad t^* = (0 \,\|\, m_0) \,\|\, t'.$$

The verifier's check on this pair evaluates to

$$F(k, 0 \,\|\, m_0) \oplus F(k, 0 \,\|\, r') \oplus F(k, 1 \,\|\, m_1) = t'.$$

This is exactly the same three terms as in the original tag, only in a different order, so the tag is valid.

If $r_0 = 1$, the adversary instead outputs

$$m^* = m_0 \,\|\, r', \qquad t^* = (1 \,\|\, m_1) \,\|\, t'.$$

Now the verifier's check evaluates to

$$F(k, 1 \,\|\, m_1) \oplus F(k, 0 \,\|\, m_0) \oplus F(k, 1 \,\|\, r') = t',$$

so this tag is valid as well.

The forged message equals the queried message only if $r' = m_0$ in the first case or $r' = m_1$ in the second case. Because r' is uniform in $\{0,1\}^{n-1}$, this happens with probability $2^{-(n-1)}$. Therefore the adversary produces a valid forgery with probability $1 - 2^{-(n-1)}$, and the scheme is insecure. ■

3. Comparing Numbers Privately (Optional)[1]

Let p be prime. Alice holds $a \in \mathbb{Z}_p$, and Bob holds $b \in \mathbb{Z}_p$. They want Alice to learn whether $a = b$ without revealing their numbers to a server, Sam. Assume that Sam follows the protocol, does not collude with either party, and has a private channel to each of them. All arithmetic below is in $\mathbb{Z}_p$.

For one equality test, Alice and Bob share independent, uniformly random keys $k_0, k_1 \in \mathbb{Z}_p$. Alice sends

$$x_a = a + k_0$$

to Sam. Bob chooses r uniformly from $\mathbb{Z}_p$ and sends Sam a pair (r, x_b). Sam computes

$$x = rx_a - x_b$$

and sends x to Alice. Alice concludes that $a = b$ if $x + k_1 = 0$.

1. Determine how Bob should compute x_b from b, r, k_0, k_1. Show that Alice's test is always correct when $a = b$, and find its error probability when $a \neq b$. Explain why Sam's view reveals nothing about a or b. Explain why, when $a \neq b$, the value $x + k_1$ reveals nothing about b to Alice.

2. Suppose Alice and Bob use the same pair (k_0, k_1) for two tests: one on inputs (a, b) and another on inputs (a', b'). Bob chooses fresh, independent values r and r' for the two tests. Show what goes wrong with privacy when the keys are reused.

3. Part 2 shows that Alice and Bob need a fresh key pair for every test. Rather than sharing many independent key pairs in advance, let $B = B(n)$ be a polynomial and suppose that they share a single long-term key $K \in \{0,1\}^n$ for a secure pseudorandom function
$$F : \{0,1\}^n \times \{0,1\}^n \longrightarrow \mathbb{Z}_p^2.$$
Design a protocol for up to $B(n)$ equality tests, and briefly justify why the resulting tests have the privacy properties of the one-time protocol.

Solution

1. Bob should send
$$x_b = r(b + k_0) + k_1.$$
Indeed,
$$\begin{aligned} x + k_1 &= r(a + k_0) - \big(r(b + k_0) + k_1\big) + k_1 \\ &= r(a - b). \end{aligned}$$
If $a = b$, then $x + k_1 = 0$ for every choice of r, so Alice's test is always correct. If $a \neq b$, then $a - b$ is nonzero in $\mathbb{Z}_p$, so
$$r(a - b) = 0 \quad \Longleftrightarrow \quad r = 0.$$

[1] This optional exercise is adapted from Dan Boneh's *A Graduate Course in Cryptography*.

Since r is uniform in $\mathbb{Z}_p$, Alice incorrectly concludes that $a = b$ with probability $1/p$.

Sam's view is (x_a, r, x_b). For fixed a and b, the map from (k_0, r, k_1) to this view is

$$(k_0, r, k_1) \longmapsto \big(a + k_0,\ r,\ r(b + k_0) + k_1\big).$$

This map is a bijection on $\mathbb{Z}_p^3$: from any view (x_a, r, x_b), the unique corresponding keys are

$$k_0 = x_a - a, \qquad k_1 = x_b - r(b + k_0).$$

Since (k_0, r, k_1) is uniform, Sam's view is uniform over $\mathbb{Z}_p^3$, independently of a and b. It therefore reveals nothing about either input.

Alice obtains $x + k_1 = r(a - b)$. If $a \neq b$, multiplication by the nonzero value $a - b$ is a permutation of $\mathbb{Z}_p$. Since r is uniform, $r(a - b)$ is uniform over $\mathbb{Z}_p$, independently of b. Thus, apart from the intended equality test, this value reveals nothing about Bob's input.

2. In the two tests, Alice sends

$$x_a = a + k_0 \qquad \text{and} \qquad x'_a = a' + k_0.$$

Sam can subtract these messages and learn

$$x_a - x'_a = a - a'.$$

Thus the reused value k_0 no longer hides the relation between Alice's two inputs.

Reusing k_1 also creates a relation between the two tests. Sam computes

$$x = r(a - b) - k_1 \qquad \text{and} \qquad x' = r'(a' - b') - k_1,$$

and can therefore obtain

$$x - x' = r(a - b) - r'(a' - b').$$

For example, if $a = b$ and $a' = b'$, this difference is always zero. With fresh independent keys, the two transcripts would instead be independently masked. Hence the one-time keys must not be reused.

3. Bob keeps a counter whose values range from 1 through $B(n)$. For each new test, he chooses the next unused value t and sends t to Alice, either directly or through Sam. The counter need not be secret. Let $\langle t \rangle_n \in \{0,1\}^n$ denote the n-bit encoding of t. Alice and Bob both compute

$$(k_0^{(t)}, k_1^{(t)}) = F_K(\langle t \rangle_n)$$

and run the one-time protocol using this pair of keys. Bob then advances his counter. Alice keeps a record of every counter value she has accepted and refuses to run a test with a repeated value or with a value outside $\{1, \ldots, B(n)\}$.

Because every test uses a distinct input to the PRF, the resulting key pairs are computationally indistinguishable from independent, uniformly random pairs in $\mathbb{Z}_p^2$. If an efficient party could distinguish the resulting protocol transcripts from those of the one-time protocol, that party could distinguish F_K from a truly random function. Thus, by the security of the PRF, the tests retain the privacy properties proved in part 1.

■